

摘要：

Jonathan Ross认为，GPU更擅长处理高度并行、对延迟不敏感的预填充阶段；LPU则凭借SRAM架构和静态调度，在低延迟、小批量的解码阶段（如MoE模型）具备极致速度与成本优势。两者的混合部署能完美平衡速度与成本。根据杰文斯悖论，算力越便宜需求越大，GPU与LPU将共同扩张市场。

Groq创始人兼CEO Jonathan Ross将英伟达GPU比作“18轮重型大货车”，将自家的LPU（语言处理单元）定位为“最后一公里配送货车”，认为两者结合才能在大语言模型推理环节实现最优的成本与速度平衡。

Jonathan Ross在近日的一次访谈中详细阐述了这一架构分工：预填充阶段（读取输入文本）高度并行、对单Token延迟不敏感，适合完全交由GPU处理；解码阶段则根据用户对速度与成本的敏感程度弹性配置，从纯GPU、GPU加LPU混合，到纯LPU不等。**他表示，LPU凭借全片上SRAM架构与静态调度机制，在低延迟、小批量的解码场景中具有显著优势，对当前主流的混合专家（MoE）模型尤为友好。**

在智能体（Agentic AI）应用快速崛起的背景下，多个AI模型相互调用的任务分解模式正推动算力需求呈指数级而非线性扩张。Jonathan Ross援引杰文斯悖论指出，算力单位成本下降不会压缩市场规模，反而会持续刺激总需求增长——GPU与LPU的市场空间本质上是共同扩张，而非零和竞争。

这也为外界理解Groq与英伟达200亿美元合作协议的战略逻辑提供了解释：在推理工作负载中，两家公司的产品承担不同角色，协同部署优于单独使用任何一方。



LPU与GPU：帕累托曲线上的互补定位

Jonathan Ross指出，GPU与LPU的每Token成本曲线形状截然不同，两者并非直接竞争关系，而是覆盖不同的性能区间。

“如果只追求最低的每Token成本，用GPU、用非常大的批量大小就行，速度会慢一些，”他说。“LPU的优势在于，能够跨多颗芯片扩展，完全依赖高速SRAM而非外部内存，在不显著抬高成本的前提下大幅提升Token生成速度。”

他表示，在帕累托曲线的高速端，LPU的经济性优于GPU；将两者组合，可以在任意目标速度下实现最优的每Token成本与最大算力容量。

LPU对混合专家（MoE）模型尤为友好。Jonathan Ross解释，GPU从DRAM读取数据时需要数百量级的批量大小才能保证经济性，而LPU仅需批量大小约10即可运行，这意味着更低的等待延迟和更高的执行效率。"LPU几乎是为专家模型量身定制的。"

静态调度与MoE：确定性架构的推理红利

Groq的另一项核心差异在于静态调度——操作顺序在编译时预先确定，而非运行时动态分配。

Jonathan Ross用日历安排打比方：短会必须精确预约，长会则可以灵活应对。"在推理场景中，你做的是超低延迟、小批量的计算，必须提前把所有操作排好，让每段计算迅速完成、及时释放硬件。训练时这不那么重要，推理时这绝对关键。"

他同时澄清，静态调度并不意味着无法适配动态路由。在MoE架构中，LPU的时间段是固定的，但"和谁开会"——即激活哪个专家的权重——是可以在运行时变化的，通过"散射和聚集"能力实现灵活路由。

与英伟达协同：预填充归GPU，解码看场景

在与英伟达达成200亿美元战略合作后，Jonathan Ross描述了两者在推理链路中的具体分工。

"预填充阶段——也就是读取输入文本的阶段——建议完全跑在GPU上，因为这个阶段高度可并行化，GPU非常擅长，"他说。解码阶段则根据用户需求分级配置：成本敏感型用户完全用GPU解码；付费专业用户采用GPU加LPU组合；极端性能场景可考虑纯LPU解码。

他预计，未来市场将看到更多LPU与GPU的混合部署形态，而非Groq芯片单独销售。**"把两者结合，就像把18轮卡车和配送货车组合使用，你能构建一个更好的网络。"**

杰文斯悖论：算力越便宜，需求越大

对于AI算力市场的长期走势，Jonathan Ross援引19世纪经济学概念"杰文斯悖论"作出判断：**算力单位成本的下降，不会压缩总需求，反而会催生更大的需求。**

"杰文斯悖论的来源是一本关于煤炭的论著：每当蒸汽机效率提升，煤炭总消耗量反而增加，"他说。"当一项活动的成本降低，之前不盈利的活动变得可行，人们愿意做更多实验。随着AI变得越来越便宜，对AI的需求只会不断增加。"

他还指出，智能体（Agent）架构将进一步放大这一效应。AI将任务拆解为并行子任务、让多个智能体同时推进，以及AI调用AI的多层嵌套模式，将导致算力使用量呈指数级扩张。"AI使用AI再使用AI，这导致了使用量的指数级爆炸。"

Jonathan Ross的结论是，"成功灾难"是不可避免的——Groq和英伟达为市场提供的算力越多，市场想要的算力就越多。



以下为访谈文字实录：

主持人： Jonathan，我们其实都是谷歌的校友。我在谷歌时，团队里有个流传的玩笑——如果当天用于在TPU上训练模型的配额用完了，不如直接放假算了。我知道你是TPU的开创者，后来离开谷歌创立了自己的芯片公司。你在谷歌看到了什么，让你想要打造一些不一样的东西？

Jonathan： 算力不够用。当时发生的事情是，语音识别团队训练了一个模型，这个模型在转录任务上超过了人类水平，那是他们第一次做到这一点。问题是，他们没办法把它投入生产。他们实际上把部署范围限定在了Nexus手机上——你应该记得，那是老款安卓手机。

主持人： 对，我用过。

Jonathan： 他们把范围限定在Nexus，与其说是作为一个功能，不如说是因为算力太少，只能支撑Nexus用户群的规模。正好在纽约，我和语音识别团队共进午餐，他们提到了这个问题。我就以20%项目的形式开始，把他们的模型移植到FPGA上，设计了一个通用架构，结果发现推理端的需求相当迫切，最后演变成了一块芯片。随后Jeff Dean做了一个分析，说“考虑到我们在这上面要投入的资金和算力规模，不如直接做ASIC算了。”我当时的反应是：能有多难？结果发现非常难——但那时我们还不知道，就这么跳进去了。

主持人： 我听你以前提过“成功灾难”这个词，我觉得这个词非常传神，在谷歌我也有过好几次这样的体验。

LPU vs. GPU：帕累托曲线与每Token成本

主持人： 英伟达GPU在训练方面表现出色，但在推理阶段存在内存瓶颈。Groq在内存架构上做了哪些改变来解决这个问题？

Jonathan： 首先要想清楚权衡取舍——没有免费的午餐。你追求的是最低的每Token成本，因为成本决定了你的算力容量。大家都在争这个——如果我花同样的钱只能得到一半的容量，那我真正关心的是每一美元能换多少Token。

当然，你同时也需要速度。权衡在于：如果你只追求最低的每Token成本，你就用GPU，用非常大的批量大小，速度会慢一些。我们用LPU做的事情，是能够跨多颗芯片扩展，不依赖任何外部内存，把模型分散在这些芯片上，从而使用速度快得多的SRAM，让Token生成更快，而成本并没有更高。

如果你了解帕累托曲线，GPU和LPU的曲线形状相当不同。在曲线的某些区间，GPU的经济性更好；在另一些区间，特别是速度更快的那端，LPU的经济性更好。把两者结合起来，就填补了



中间地带。GPU、GPU加LPU的组合、以及纯LPU，三者加在一起，在任何你想要的速度下，都能实现最优的每Token成本和最大的算力容量。

静态调度与混合专家模型

主持人： Groq的另一个差异化之处是静态调度——操作顺序在编译时就已预先确定。这对大语言模型推理来说有什么优势？

Jonathan： 我用日历安排来打比方。如果我要进行一堆15分钟的短会，我必须提前排好日程，因为对方必须准时出现。但如果是一个5小时的长会，就不需要那么精确了——你来了就聊，迟到30分钟也不过是5小时里的一小部分。

在推理场景中，你在做的是超低延迟、小批量的计算，所以你需要把所有操作都预先排好，让每一段计算能迅速完成，及时释放硬件供下一步使用，不至于让后续的所有工作都在那里等。训练时这不那么重要，推理时这绝对关键。

主持人： 当今最前沿的大语言模型大多采用混合专家架构，推理时每个查询可能激活不同的专家子集。这在一块采用静态调度的芯片上是怎么运作的？

Jonathan： 关键在于什么东西被静态调度了。在LPU上，我把这个15分钟的时间段排好了，但和谁开会是可以变的。LPU有做"散射和聚集"的能力，意味着根据需要激活哪个专家，我们会去取不同的专家权重。运行时间仍然一样，只是换了一个不同的专家。如果专家的大小不同，我们甚至可以路由到另一颗芯片，当然流水线里会有短暂的气泡，但确定性给了你更强的预测时序的能力，而不会限制你能运行什么。

而且LPU架构对混合专家模型特别有利，因为批量大小越小越好——而混合专家在批量大小上天然处于不利地位：从DRAM读取数据时，你需要很大的批量（可能数百个）才能让经济账算过来；而在LPU上，批量大小只需要10左右就能跑通，这意味着你不需要等那么多查询积累起来才能执行，从而降低了延迟，提升了效率。LPU几乎是为专家模型量身定制的。

自回归与扩散模型

主持人： 说到架构，当Transformer被下一代架构取代时，LPU需要完全重新设计，还是它与当前大语言模型的形态是正交的？

Jonathan： 这是个经典问题。LPU设计的时候，《Attention Is All You Need》那篇论文还没有发表。注意力机制和当时已有的一些架构（比如卷积）有很多相似之处，虽然两者相当不同，但归根结底都是线性代数。如果你为线性代数构建了一块最优的芯片，你就为大多数这类架构构建了一块最优的芯片。

你可以选择针对特定的矩阵乘法大小做优化，不同架构可能有差异。我见过一些人尝试极度专用化，但最终赢得最多的几乎每次都是灵活性。打个比方：如果我告诉你，我能让运行速度快10倍，但代价是你永远不能再改变模型，你会接受吗？答案大概是不会——因为算法本身可能就会有10倍的改进。最近就有一个改变注意力机制工作方式的进展，把规模缩小了10倍。算法改进的速度非常快，灵活性往往比优化本身更重要。

LPU架构在设计上特别注重易于编程，使得新架构出现时能被快速采用，最新的算法可以很快上线运行。

主持人： LPU中的L代表"语言"——这是否意味着视觉和音频模型无法从同样的加速中受益？

Jonathan： Groq云目前最大的用户群之一是语音转文字的用户，我们也做过一段时间的文字转语音，原因是这类任务对实时性极其敏感。很多语音模型里面还嵌有卷积层之类的东西，这正是



通用架构的价值所在——否则这些语音任务根本没办法在上面跑。

更有意思的是，速度更快实际上还能提升质量，这有点反直觉。音频处理可以把音频切成非常小的片段来处理，但如果每次只听一小段，你就缺乏完整的上下文，预测词语就更难了。用较慢的芯片做音频处理时，为了满足实时性要求，只能切成更小的片段，而这会提高错误率——就像让两个人同时转录一段演讲，但每人每次只能听5秒，错误率会大幅上升。LPU能以数百倍于实时速度的速度完成语音转录，所以可以处理大得多的片段，从而降低这些模型的错误率。

主持人：我们谈到的这些应用场景，语言推理和音频，大多是自回归的；而现在的视觉模型很多是基于扩散的，一些大语言模型也在用扩散架构。扩散大语言模型在GPU上比自回归大语言模型快很多，这个排名在Groq芯片上还成立吗？

Jonathan：扩散模型受益于总算力量的大小。先给大家解释一下什么是自回归——自回归简单来说就是：先推断出第一个词，再推断下一个词，就像下棋，我先想好这一步再走下一步，而不是一次性预判所有步骤。在语言中，要知道第100个词是什么，通常要先知道第99个词是什么。

当然你可以做一些分解：某些词比其他词更重要，先预测重要的词，再在周围填入其他词。

我看到很多人在尝试用扩散模型生成语言，但效果不太好。原因在于，在你还没决定这里要说什么之前，很难确定那里要说什么。这和之前提到的音频切片问题一样——想象100个人同时写一篇演讲稿，每人都看不到其他人写的内容。扩散之所以叫“扩散”，是因为信息在时间和空间中扩散，距离越远，影响越弱。

从质量角度来看：如果你用自回归与扩散分别生成音乐，自回归版本会更有灵魂，更有深度，你会更喜欢，但可能有一两声杂音；纯扩散版本则是你听过的最纯净的电梯音乐，完全没有灵魂。但如果把两者结合起来，重要的音乐节点用自回归加上下文来处理，其余的用扩散来填充，效果就会大不相同。

就像我们把LPU和GPU组合起来用于大语言模型的解码一样，我认为扩散大语言模型最终成功的版本，很可能也是把自回归和扩散结合起来的。

Groq与英伟达Vera Rubin的协同

主持人：英伟达在今年3月的GTC大会上发布了专为推理（尤其是智能体场景）设计的Vera Rubin超级计算机。GPU和Groq在推理时是如何协同工作的？

Jonathan：我来打个比方。假设你要为整个美国建立一套物流网络，从零开始，你可以选择18轮卡车或者配送货车。配送货车可以进任何车道，但装载量小，单位成本更贵。最优解是两者都要。

在这个比喻中，GPU是18轮卡车——能一次处理大量Token，但装载和运输需要一些时间；**LPU更像配送货车——效率不如前者，但在“最后一公里”比那个庞然大物更高效。**就像之前谈到的混合专家模型，LPU在其中某些部分上有优势。把两者结合起来，就像把18轮卡车和配送货车组合使用，你能构建一个更好的网络。

大语言模型的推理分两个独立部分：权重层和注意力层。我们的方案是把投影层放在LPU上，把注意力层放在GPU上，两者各取所长。

主持人：英伟达合作协议之后，我们应该预期Groq芯片继续独立销售，还是会看到更多LPU加GPU的混合形态？

Jonathan：我认为你会看到更多混合形态。预填充阶段——也就是读取文本的阶段——我们仍然建议完全跑在GPU上，因为GPU非常擅长这个，而且这个阶段对每Token的延迟不那么敏



感，是高度可并行化的，丢给GPU这辆18轮卡车就行。

解码阶段则要看情况：对于成本敏感的应用，比如免费用户，可能会完全用GPU做解码；如果是付费的专业用户，他们对速度有更高要求，大概率会用GPU加LPU的组合；对于极端追求性能的任务，甚至可能纯用LPU做解码。总体而言，任何数据中心的配置都是：预填充完全在GPU上，解码部分在LPU上、部分在GPU上。

智能体推理与规模经济

主持人： Vera Rubin超级计算机主要针对智能体推理场景。过去一年，智能体应用迅速崛起，这如何改变了规模化推理的单位经济和成本？

Jonathan： 首先，我认为大多数人并不真正理解“智能体”是什么，只是把这个词当成流行语在用。让我来真正解释一下，因为这非常重要。

智能体有点像AI领域的英伟达——它的核心是能够把任务拆解成并行的子任务。CPU是串行的，GPU是并行的。如果你一个人完成一项任务，你只能一次做一件事，还会被各种等待卡住，效率不高。但如果你能把任务拆开，就能让多人同时推进。AI也面临类似的瓶颈——我们前面谈到，在生成第99个Token之前无法生成第100个Token，但如果你能把问题拆解成没有这种依赖关系的子任务，就可以让多个智能体、多个上下文窗口同时工作。对于大多数问题，这是可行的。

还有另一个层面：AI使用AI。就像你在为采访做准备时会用AI来帮你准备问题，AI也会向另一个AI提问，让它在后台处理，等结果回来再整合进自己的答案。任务分解给AI、AI再分发给其他AI，AI使用AI再使用AI，这导致了使用量的指数级爆炸。而且答案质量往往随着并行子任务数量的增加而提升，因为就像团队规模更大、能做更多交叉验证一样，最终的答案更有依据。

AI能否替代CUDA内核工程师？

主持人： CUDA内核手工编写难度极大。你认为AI是否已经能够自己写出来？

Jonathan： 我认为也许已经足够好了，但这个问题不是非黑即白的。“足够好”意味着什么？你不是“写内核”或“不写内核”这种二选一的问题——关键在于这个内核有多好？效率如何？性能如何？与其他内核的融合是否容易？通用性如何？可复用性如何？随着AI能力不断提升，内核质量会持续提升，而你在一个特定内核上花的时间越多，这个内核就会越好。

有趣的是，Groq架构——LPU——实际是无内核架构。当初设计的时候，我们还没有大语言模型可以帮我们写软件，只能全靠自己，而且团队规模小，所以我们构建了一块编译复杂度很低的芯片。就像AI随着时间推移会生成越来越好的内核一样，如果它要编译的硬件越容易理解，它生成的内核就会更好。我们已经在用AI来编程LPU，效果很好，因为这个问题对大语言模型来说很容易“脑补”。

主持人： AI降低了写软件的门槛，从你说的来看，硬件方面也开始出现同样的趋势。我们会看到更多人因为门槛降低而去做硬件吗？

Jonathan： 绝对会。你会看到更多人尝试设计硬件。但我认为有一个问题——硬件是物理的东西，需要做实验。软件开发的成果是即时反馈，可以快速迭代；硬件有供应链，有大额赌注。你会看到很多人去做芯片，因为设计一颗芯片会变得很容易，但把它推向量产非常难。这会变成“小海龟问题”——世界上的供应资源是有限的，客户在下注时，会选择他们知道靠得住的。

大语言模型让写软件、写RTL（芯片的编程语言）变得更容易，做的人会更多，但真正能走到量产的，反而可能更少，因为太难取舍了——客户只想押注在能依赖的公司身上。



主持人： 这和软件领域其实很像——在卧室里做出原型很容易，但真正推向市场并保证可靠性就难多了。

Jonathan： 有一个关键区别。软件发现了Bug，你可以打补丁。芯片出了错，首先需要4到6个月重新流片。芯片是物理的东西，制造时要经历60到70层的化学沉积，每层可能需要一天甚至更长时间，从你完成“流片”（即提交芯片掩膜版）到拿到可以测试的实物，有固定的物理时间。这个掩膜版本身就要几千万美元，做错了就损失几千万。但这和告诉客户“抱歉，还需要再等六个月才能拿到产品，我要做修改”相比，后者的代价更大。更何況，供应链的运作方式要求你必须先买晶圆，提前做出承诺——如果到时候没有可交付的芯片，代价极为惨重。

所以我不认为你会看到那种“大家都来乱扔芯片”的情况，而是会看到很多规模较小的玩家做芯片，但胜出的只会是少数，因为赌注太高，客户只会选择可以依赖的对象，尤其是随着成本不断攀升。

主持人： AI是否在一些外行人想不到的方面让硬件设计变得更容易？

Jonathan： 有个很有意思的现象。我们注意到，过去硬件工程师从来不自己写软件，遇到需要写软件的时候，都会去找软件工程师帮忙。但现在他们开始说：“我直接实现一个小的软件测试，看看这个设计是否合理。”然后他们立刻得到反馈，发现“哦，这里用起来比我想的要难”。

硬件和软件开发本来泾渭分明，两个领域有很多相似之处，但语言不同，思维方式也有微妙差别——做芯片时要考虑导线、门电路等物理约束，这让硬件工程师对写软件有一种畏惧，软件工程师对硬件也是一样。但现在，一个硬件工程师只需要让大语言模型写一段软件跑在自己的硬件上，发现跑不顺，就能意识到哪里需要改进。AI让这种跨学科的自助服务成为了现实，以前这些分离的学科之间有清晰的界限，现在界限开始模糊了，大家能够伸手到相邻的学科里做一些事情。

主持人： 这和我们看到的软件工程师和设计师之间的变化一样——软件工程师不需要等设计稿就能实现东西，很多设计师也在用代码工具直接把想法做出来。

Jonathan： 是的，而且如果软件工程师和硬件工程师之间有争论，现在可以直接实现出来证明对方。

杰文斯悖论：算力越便宜，需求越大

主持人： 我们从谷歌的“成功灾难”聊起，那你希望Groq和英伟达未来发生哪些“成功灾难”？

Jonathan： 这里要说到杰文斯悖论——算力的需求是无限的。只要文明还有未解决的问题，我们就需要更多算力。现在癌症还没被治愈，人还会变老，而且算力还不够，这就是三个现成的问题。只要这些问题存在，我们就必须继续向前推进。

这意味着我们需要更智能的AI，需要更多算力让更多AI并行运行、并行解决更多问题。随着我们不断进步，单位智能的成本会下降，然后就进入了杰文斯悖论——成本越低，人们越愿意花。

杰文斯悖论的来源是19世纪的一本关于煤炭的论著：作者发现，每当蒸汽机效率提升，煤炭总消耗量反而增加。原因是，当一项活动的成本降低，之前不盈利的活动变得盈利可行，人们愿意做更多这件事，愿意做更多实验、尝试更多东西。随着AI变得越来越便宜，对AI的需求会不断增加，直到人们在AI上的支出越来越多，他们需要的算力也越来越多。

还有一个类比：如果你从地下抽出两倍的石油，并不意味着两倍的人能得到交通便利，因为还需要有车。但一旦你训练好一个模型，多提供两倍的算力，就能让两倍的人使用它，解决两倍的问题。每建起一座AI工厂，你立刻就能做更多的事，这会促使人们想做更多事，进而不断压低成



本，杰文斯悖论也就一直在发挥作用。所以"成功灾难"是不可避免的——我们为世界提供的算力越多，人们想要的算力就越多。

AI时代应该培养什么能力？

主持人：最后，你有什么想对这群技术敏锐、充满好奇心的听众说的？

Jonathan：很多人来问我，孩子应该学什么？我的答案很简单。我们今天的教育方式基于信息时代的思维——教孩子回答问题，给出答案。但有了AI之后，这件事翻转了——变成了如何提出正确的问题。如果你能提出正确的问题，AI就能替你去找答案。

所以我给所有听众最大的建议是：开始学习如何问出更好的问题。教会你的孩子如何问出更好的问题。教育体系需要重构，以"提问"为导向。

如果孩子们把你的问题输入AI就能很容易地解决，那你并没有在教他们如何在未来取得成功。但如果你给他们一个需要自己想出问题的挑战，你才是在真正为他们的未来做准备。

主持人：这说得很有道理。我曾经从研究和创业的生活中休息了一段时间，因为我发现直接和AI对话、问问题、学新东西，让我感到极大的乐趣。我制作这个视频的方式，就是借助AI边学硬件边做——我可以问那些永远不会出现在论文里的"为什么不这样做"的问题。非常感谢你今天的到来，真是一次非常愉快的对话。

Jonathan：谢谢邀请。

风险提示及免责声明

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

* 体验资格每人限申领一次

扫码

免费领取

VIP会员·7天畅读卡 | 大师课·入门串讲



限免

55 收藏

分享到:

写评论

请发表您的评论

表情 图片

发表评论

1 条评论





见闻用户

13小时前 中国北京

LPU FFN vs AMX FFN

👍 0 ↩ 回复

